

Федеральное государственное автономное образовательное учреждение высшего
образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки 09.03.04 «Программная инженерия» –

Системное и прикладное программное обеспечение

Отчёт

По лабораторной работе №2

Проектирование вычислительных систем

Вариант: 3

Выполнили:

студенты 3 курса

Батманов Даниил Евгеньевич

Разинкин Александр Владимирович

Группа: Р3307

Принял:

Пинкевич Василий Юрьевич

Отчёт принят «__»_____2025 г.

Оценка: _____

г. Санкт-Петербург, 2025

Цели работы

1. Изучить протокол передачи данных по интерфейсу UART.
2. Получить базовые знания об организации системы прерываний в микроконтроллерах на примере микроконтроллера STM32.
3. Изучить устройство и принципы работы контроллера интерфейса UART, получить навыки организации обмена данными по UART в режимах опроса и прерываний.

Задание

Разработать и реализовать два варианта драйверов UART для стенда SDK-1.1M: с использованием и без использования прерываний. Драйверы, использующие прерывания, должны обеспечивать работу в «неблокирующем» режиме (возврат из функции происходит сразу же, без ожидания окончания приема/отправки), а также буферизацию данных для исключения случайной потери данных. В драйвере, не использующем прерывания, функция 77 приема данных также должна быть «неблокирующей», то есть она не должна зависать до приема данных (которые могут никогда не поступить). При использовании режима «без прерываний» прерывания от соответствующего блока UART должны быть запрещены.

Написать с использованием разработанных драйверов программу, которая выполняет определенную вариантом задачу. Для всех вариантов должно быть реализовано два режима работы программы: с использованием и без использования прерываний. Каждый принимаемый стендом символ должен отсылаться обратно, чтобы он был выведен в консоли (так называемое «эхо»). Каждое новое сообщение от стенда должно выводиться с новой строки. Если вариант предусматривает работу с командами, то на каждую команду должен выводиться ответ, определенный в задании или «ОК», если ответ не требуется. Если введена команда, которая не поддерживается, должно быть выведено сообщение об этом.

Скорость работы интерфейса UART должна соответствовать указанной в варианте задания.

Порядок выполнения работы

1. Изучить:

– разделы учебного пособия:

- 1.3. Прерывания;
- 1.4. Последовательный интерфейс UART;

– электрическую принципиальную схему стенда в части назначения сигналов отладочного интерфейса UART;

– раздел справочного руководства RM0090: «Universal synchronous asynchronous receiver transmitter (USART)», знать устройство контроллера USART, режимы его работы и способы настройки;

- раздел справочного руководства PM0214 (programming manual) [43] о контроллере прерываний: «Nested vectored interrupt controller (NVIC)», знать принципы организации системы прерываний в микроконтроллере STM32F4 и приемы ее использования;
 - состав стандартного драйвера USART из библиотеки HAL и содержимое создаваемых генератором файлов usart.c/.h.
2. Подготовить шаблон проекта для STM32CubeIDE, настроить тактовые частоты.
 3. Задать настройки контроллера UART, настроить входы и выходы микроконтроллера.
 4. Разработать драйверы UART для работы в режимах опроса и прерывания. Драйвер, работающий в режиме прерывания, должен поддерживать буферизацию данных и работу в «неблокирующем режиме».
 5. Разработать программу согласно варианту задания и провести ее тестирование.

Ход работы

Вариант 3

Доработать программу, посылающую сигналы азбуки Морзе для латинского алфавита. Последовательность точек и тире, полученных нажатием кнопки стенда, должна дешифроваться и отправляться через последовательный канал в виде символов (букв латинского алфавита).

Символы, получаемые стендом через последовательный канал, должны шифроваться и «проигрываться» с помощью светодиода. Если в момент «мигания» приходят новые символы, они добавляются в очередь. Должна быть возможность одновременной буферизации до восьми символов: они должны запоминаться стендом и последовательно проигрываться.

Считывание нажатий кнопки и отправка дешифрованных символов должна функционировать одновременно с приёмом через последовательный канал и миганием светодиода.

Включение/выключение прерываний должно осуществляться при получении стендом символа «+».

Скорость обмена данными по UART – 115200 бит/с

Настройки модуля USART-6

Mode

Mode Asynchronous

Configuration

Reset Configuration

NVIC Settings	DMA Settings	GPIO Settings
Parameter Settings		User Constants

Configure the below parameters :

⏪ ⏩ ⓘ

Basic Parameters

Baud Rate	115200 Bits/s
Word Length	8 Bits (including Parity)
Parity	None
Stop Bits	1

Advanced Parameters

Data Direction	Receive and Transmit
Over Sampling	16 Samples

NVIC Settings	DMA Settings	GPIO Settings
Parameter Settings		User Constants

NVIC Interrupt Table	Enabled	Preemption Priority
USART6 global interrupt		0

Исходный код

```
/* USER CODE BEGIN Header */
/**
```

* * * *

```
* @file      : main.c
```

* *@brief* : Main program body

* @attention

*

* Copyright (c) 2025 STMicroelectronics.

* All rights reserved.

*

* This software is licensed under terms that can be found in the LICENSE file

* in the root directory of this software component.

* If no LICENSE file comes with this software, it is provided AS-IS.

*

*/

/* USER CODE END Header */

/* Includes -----*/

#include "main.h"

#include "usart.h"

#include "gpio.h"

/* Private includes -----*/

/* USER CODE BEGIN Includes */

/* USER CODE END Includes */

/* Private typedef -----*/

/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define -----*/

/* USER CODE BEGIN PD */

/* Константы для работы с индикатором */

#define SHORT_SIGNAL_DURATION 200

#define LONG_SIGNAL_DURATION 500

```
/* Константы для считывателя кнопки */
```

```
#define ANTI_BOUNCE_DELAY 40  
#define LONG_PRESS_DELAY 400  
#define VERY_LONG_PRESS_DELAY 3000
```

```
/* Константы для очередей */
```

```
#define QUEUE_SIZE 10
```

```
/* Константы для UART */
```

```
#define UART_TIMEOUT 100
```

```
/* Константы для Морзе */
```

```
#define MAX_MORZE_LENGTH 5
```

```
/* USER CODE END PD */
```

```
/* Private macro -----*/
```

```
/* USER CODE BEGIN PM */
```

```
/* USER CODE END PM */
```

```
/* Private variables -----*/
```

```
/* USER CODE BEGIN PV */
```

```
typedef enum {  
    IN_PROGRESS,  
    NOT_PRESSED,  
    SHORTLY_PRESSED,  
    LONGLY_PRESSED,  
    VERY_LONGLY_PRESSED  
} button_state;
```

```
typedef enum {
```

```

    ENABLED,
    DISABLED
} interruption_mode;

interruption_mode current_mode = DISABLED;

char *interruption_disabled_message = "Interruption disabled\n";
uint32_t interruption_disabled_message_size = 22;
char *interruption_enabled_message = "Interruption enabled\n";
uint32_t interruption_enabled_message_size = 21;

typedef struct {
    uint8_t items[QUEUE_SIZE];
    uint32_t front;
    uint32_t rear;
} queue;

queue read_queue;
uint8_t read_buffer;

uint8_t write_buffer[MAX_MORZE_LENGTH];
uint8_t morze_length = 0;

typedef struct {
    uint8_t symbol_ascii;
    uint8_t *symbol_morze;
} symbol_morze;

symbol_morze known_symbol_morze[] = {
    { 'A', (uint8_t*)".-" },
    { 'B', (uint8_t*)"-..." },
    { 'C', (uint8_t*)"-.-" },
    { 'D', (uint8_t*)"-.." },
    { 'E', (uint8_t*)"." },
    { 'F', (uint8_t*)"..-." },
    { 'G', (uint8_t*)"--." },
    { 'H', (uint8_t*)"...." },
    { 'I', (uint8_t*)".." },
    { 'J', (uint8_t*)".---" },

```



```

{ 'K', (uint8_t*)"-.-" },
{ 'L', (uint8_t*)"-..." },
{ 'M', (uint8_t*)"--" },
{ 'N', (uint8_t*)"-. " },
{ 'O', (uint8_t*)"---" },
{ 'P', (uint8_t*)"-. ." },
{ 'Q', (uint8_t*)"--. " },
{ 'R', (uint8_t*)"-. " },
{ 'S', (uint8_t*)"..." },
{ 'T', (uint8_t*)"-" },
{ 'U', (uint8_t*)"..-" },
{ 'V', (uint8_t*)"...-" },
{ 'W', (uint8_t*)"--" },
{ 'X', (uint8_t*)"-. ." },
{ 'Y', (uint8_t*)"-. ." },
{ 'Z', (uint8_t*)"--.." },

{ '0', (uint8_t*)"-----" },
{ '1', (uint8_t*)" .----" },
{ '2', (uint8_t*)" ..---" },
{ '3', (uint8_t*)" ...--" },
{ '4', (uint8_t*)" ....-" },
{ '5', (uint8_t*)" ..... " },
{ '6', (uint8_t*)" -...." },
{ '7', (uint8_t*)" --..." },
{ '8', (uint8_t*)" ----.." },
{ '9', (uint8_t*)" ----." }

};

/* USER CODE END PV */

/* Private function prototypes -----*/
void SystemClock_Config(void);
/* USER CODE BEGIN PFP */

void send_mode_changed_message();

button_state check_button();
button_state process_input();

```

```

void init_queue(queue*);
_Bool enqueue(queue*, uint8_t);
_Bool dequeue(queue*, uint8_t*);

void flash_indicator(uint16_t, char);
void play_symbol(uint8_t symbol);
void send_morze();

void print_message(int num);

/* USER CODE END PFP */

/* Private user code -----*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{
    /* USER CODE BEGIN 1 */

    /* USER CODE END 1 */

    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    HAL_Init();

    /* USER CODE BEGIN Init */

    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

```

```

/* USER CODE BEGIN SysInit */

/* USER CODE END SysInit */

/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_USART6_UART_Init();
/* USER CODE BEGIN 2 */

init_queue(&read_queue);

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */

HAL_UART_Receive_IT(&huart6, &read_buffer, 1);
while (1)
{
    HAL_NVIC_EnableIRQ(USART6_IRQn);
    if (current_mode == DISABLED) {
        if (HAL_UART_Receive(&huart6, &read_buffer, 1, UART_TIMEOUT) ==
HAL_OK) {
            enqueue(&read_queue, read_buffer);
        }
    }

    if (dequeue(&read_queue, &read_buffer)) {
        if (read_buffer == '+') {
            if (current_mode == DISABLED) {
                HAL_NVIC_EnableIRQ(USART6_IRQn);
                HAL_UART_Receive_IT(&huart6, &read_buffer, 1);
                current_mode = ENABLED;
            } else {
                HAL_UART_Abort(&huart6);
                HAL_NVIC_DisableIRQ(USART6_IRQn);
                current_mode = DISABLED;
            }
        }
    }
}

```

```

        send_mode_changed_message();
    } else {
        play_symbol(read_buffer);
    }
}

if (process_input() == VERY_LONGLY_PRESSED || morze_length == 5) {
    send_morze();
    morze_length = 0;
}

/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /** Configure the main internal regulator output voltage
     */
    __HAL_RCC_PWR_CLK_ENABLE();

    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE
3);

    /** Initializes the RCC Oscillators according to the specified parameters
     * in the RCC_OscInitTypeDef structure.
     */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;

```

```

RCC_OscInitStruct.HSISState = RCC_HSI_ON;
RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
{
    Error_Handler();
}

/** Initializes the CPU, AHB and APB buses clocks
 */
RCC_ClkInitStruct.ClockType =
RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
        |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_HSI;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;

if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
{
    Error_Handler();
}
}

/* USER CODE BEGIN 4 */
void print_message(int num) {
    char buf[128];
    sprintf( buf, "Number: %d\n", num );
    HAL_UART_Transmit( &huart6, (uint8_t *) buf, strlen( buf ), 100);
}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart == &huart6) {
        enqueue(&read_queue, read_buffer);
        HAL_UART_Receive_IT(&huart6, &read_buffer, 1);
    }
}

void wait(uint32_t timeout) {

```

```

uint32_t start_time = HAL_GetTick();
while (HAL_GetTick() - start_time < timeout) {
    if ((current_mode == DISABLED) && (HAL_UART_Receive(&huart6,
&read_buffer, 1, timeout / 10) == HAL_OK)) {
        enqueue(&read_queue, read_buffer);
    }
}
}

```

```

void send_mode_changed_message() {
    if (current_mode == DISABLED) {
        HAL_UART_Transmit(&huart6, interruption_disabled_message,
strlen((char*)interruption_disabled_message), 100);
    } else {
        HAL_UART_Transmit_IT(&huart6, interruption_enabled_message,
strlen((char*)interruption_enabled_message));
    }
}

```

```

button_state check_button() {
    static _Bool was_pressed = 0;
    static uint32_t start_time = 0;

    if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_15) == GPIO_PIN_RESET) { /*
Кнопка на текущий момент нажата */
        if (!was_pressed) {
            was_pressed = 1;
            start_time = HAL_GetTick();
        }

        return IN_PROGRESS;
    } else { /* Кнопка на текущий момент не нажата */
        if (!was_pressed) {
            return NOT_PRESSED;
        }
        was_pressed = 0;

        uint32_t current_delay = HAL_GetTick() - start_time;
        if (current_delay < LONG_PRESS_DELAY) {

```

```

        return SHORTLY_PRESSED;
    } else if (current_delay >= LONG_PRESS_DELAY && current_delay <
VERY_LONG_PRESS_DELAY) {
        return LONGLY_PRESSED;
    } else {
        return VERY_LONGLY_PRESSED;
    }
}
}

```

```

button_state process_input() {
    button_state state = check_button();

    if (state == IN_PROGRESS || state == NOT_PRESSED) {
        return state;
    } else if (state == SHORTLY_PRESSED) {
        write_buffer[morze_length++] = '.';
        flash_indicator(GPIO_PIN_14, '.');
    } else if (state == LONGLY_PRESSED) {
        write_buffer[morze_length++] = '-';
        flash_indicator(GPIO_PIN_15, '-');
    } else {
        return state;
    }
}
}

```

/* Работа с очередью */

```

void init_queue(queue *q) {
    q->front = -1;
    q->rear = -1;
}

```

```

_Bool is_empty(queue *q) {
    return q->front == -1;
}

```

```

_Bool is_full(queue *q) {
    return (q->rear + 1) % QUEUE_SIZE == q->front;
}

```

```

}

_Bool enqueue(queue *q, uint8_t value) {
    // Запрещаем прерывания при работе с очередью
    uint32_t pmask = __get_PRIMASK();
    // HAL_NVIC_DisableIRQ(USART6_IRQn);
    if (is_full(q)) {
        return 0;
    }

    if (is_empty(q)) {
        q->front = 0;
        q->rear = 0;
    } else {
        q->rear = (q->rear + 1) % QUEUE_SIZE;
    }

    q->items[q->rear] = value;
    __set_PRIMASK(pmask);
    return 1;
}

_Bool dequeue(queue *q, uint8_t *value) {
    // Запрещаем прерывания при работе с очередью
    uint32_t pmask = __get_PRIMASK();
    // HAL_NVIC_DisableIRQ(USART6_IRQn);
    if (is_empty(q)) {
        return 0;
    }

    *value = q->items[q->front];

    if (q->front == q->rear) {
        q->front = -1;
        q->rear = -1;
    } else {
        q->front = (q->front + 1) % QUEUE_SIZE;
    }
}

```



```

__set_PRIMASK(pmask);
return 1;
}

```

/* Работа с индикатором */

```

void flash_indicator(uint16_t pin, char morze_symbol) {
    HAL_GPIO_WritePin(GPIOD, pin, GPIO_PIN_SET);
    if (morze_symbol == '.') {
        wait(SHORT_SIGNAL_DURATION);
    } else if (morze_symbol == '-') {
        wait(LONG_SIGNAL_DURATION);
    }
    HAL_GPIO_WritePin(GPIOD, pin, GPIO_PIN_RESET);
}

```

```

void play_symbol(uint8_t symbol) {
    int array_size = sizeof(known_symbol_morze) / sizeof(symbol_morze);
    for (int i = 0; i < array_size; i++) {
        symbol_morze *sm = &known_symbol_morze[i];
        if (symbol == sm->symbol_ascii) {
            uint8_t *morze = sm->symbol_morze;
            for (int j = 0; morze[j] != '\0'; j++) {
                flash_indicator(GPIO_PIN_13, morze[j]);
                wait(100); // пауза между точками/тире одного символа
            }
            break;
        }
    }
}

```

```

void send_morze() {
    int array_size = sizeof(known_symbol_morze) / sizeof(symbol_morze);

    for (int i = 0; i < array_size; i++) {
        symbol_morze *sm = &known_symbol_morze[i];
        uint8_t *morze_code = sm->symbol_morze;

        // Проверяем длину морзе-кода
    }
}

```

```

int code_length = 0;
while (morze_code[code_length] != '\0' && code_length < MAX_MORZE_LENGTH)
{
    code_length++;
}

// Если длина не совпадает, пропускаем
if (code_length != morze_length) {
    continue;
}

// Сравниваем символы морзе-кода
_Bool match = 1;
for (int j = 0; j < morze_length; j++) {
    if (write_buffer[j] != morze_code[j]) {
        match = 0;
        break;
    }
}

// Если нашли совпадение, отправляем ASCII символ
if (match) {
    uint8_t symbol_to_send = sm->symbol_ascii;
    HAL_UART_Transmit(&huart6, &symbol_to_send, 1, 100);
    return; // Выходим после отправки
}

// Если символ не найден, отправляем '?' или другой символ ошибки
uint8_t unknown_symbol = '?';
HAL_UART_Transmit(&huart6, &unknown_symbol, 1, 100);
}

/* USER CODE END 4 */

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None

```

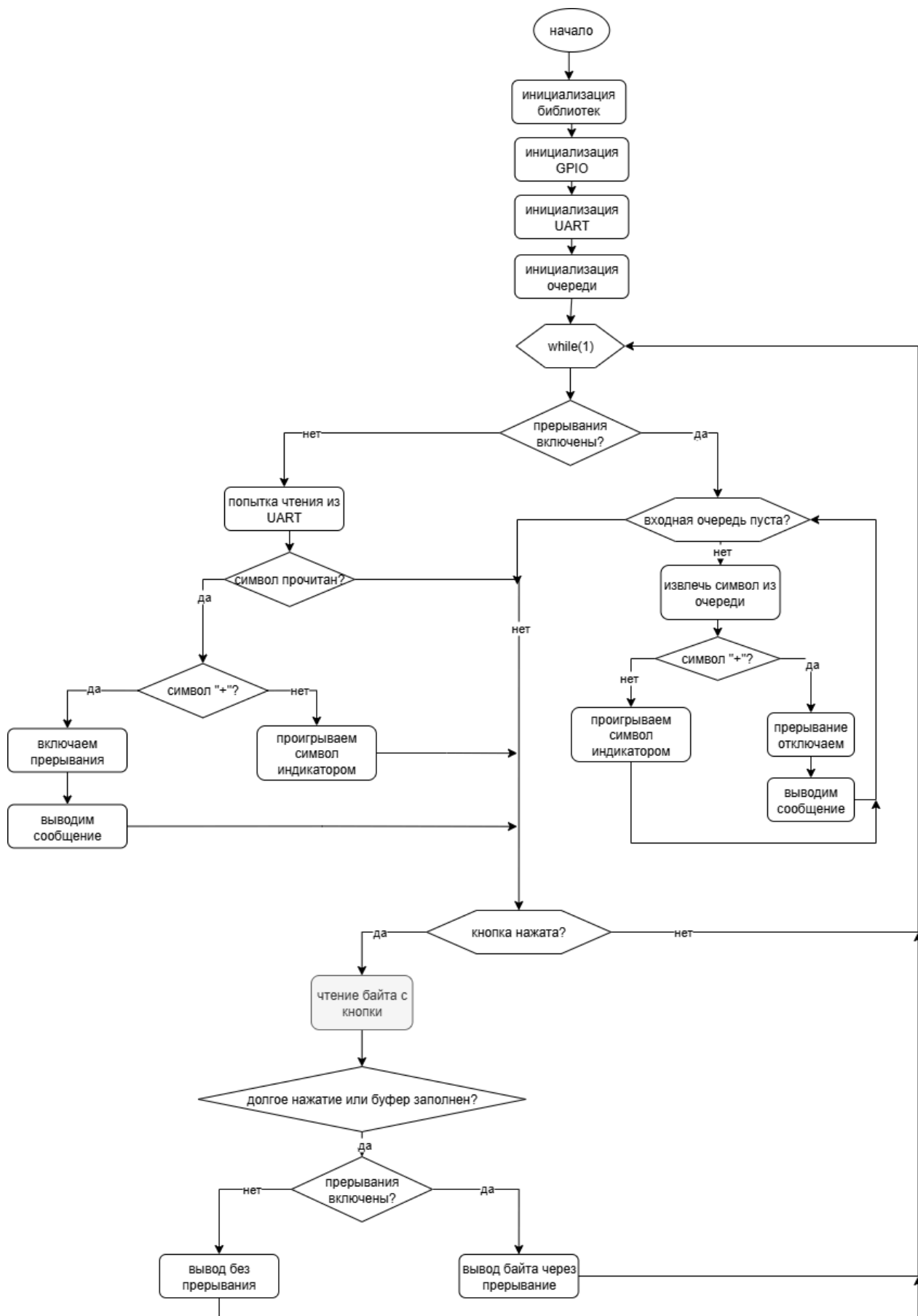
```

*/
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1)
    {
    }
    /* USER CODE END Error_Handler_Debug */
}

#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number
 *        where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and line number,
       ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
    /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */

```

Блок-схема



Заключение

В ходе выполнения лабораторной работы удалось доработать программу, посылающую сигналы азбуки Морзе для латинского алфавита. Удалось притронуться к прерываниям, реализовать работу алгоритму при включенных и выключенных прерываниях.